

Design Pattern

Pertemuan 12

Dosen Pengampu:
Zulkifli Arsyad, S.Kom., M.T.
Joe Lian Min, M.Eng
Muhammad Riza Alifi, S.T., M.T.



Topik

- Strategy Pattern
- State Pattern
- Factory Pattern
- Composite Pattern
- Decorator Pattern

Studi kasus domain P2P Lending



INTRODUCTION TO DESIGN PATTERN

Design Pattern adalah **solusi umum** untuk masalah desain software yang sering muncul.

TUJUAN DESIGN PATTERN



Membuat kode fleksibel



Mudah dikembangkan



Mudah dirawat

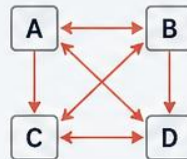


Mengurangi coupling



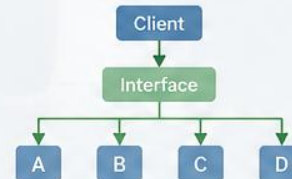
Mendukung perubahan bisnis

TANPA DESIGN PATTERN



Ketergantungan tinggi, sulit diubah dan dirawat

DENGAN DESIGN PATTERN



Ketergantungan rendah, mudah diubah dan dikembangkan

MASALAH YANG SERING MUNCUL

- ✗ Kode sulit diubah
- ✗ Sulit dikembangkan
- ✗ Banyak duplikasi
- ✗ Ketergantungan tinggi (coupling)
- ✗ Perubahan kebutuhan bisnis berdampak besar

MENGAPA DESIGN PATTERN PENTING?



Reusability
Solusi yang sudah teruji dan terbukti efektif



Best Practice
Mengikuti praktik terbaik dari para ahli



Komunikasi Lebih Baik
Menyediakan bahasa bersama bagi tim developer



Produktivitas Meningkat
Pengembangan lebih cepat, code lebih berkualitas



Intinya: Design Pattern bukan kode jadi, tapi **cetak biru (blueprint)** untuk menyelesaikan masalah desain yang berulang.

DESIGN PATTERN GoF

Gang of Four (GoF) mengkategorikan 23 pola desain menjadi 3 kelompok besar

1. CREATIONAL PATTERNS (Cara membuat objek)



Singleton



Factory Method



Abstract Factory



Builder



Prototype

2. STRUCTURAL PATTERNS (Cara menyusun struktur)



Adapter



Bridge



Composite



Decorator



Facade



Flyweight



Proxy

3. BEHAVIORAL PATTERNS (Cara mengelola perilaku)



Observer



Strategy



Command



Iterator



Mediator



Memento



State



Visitor



Template Method



Chain of Responsibility

KAPAN MENGGUNAKAN DESIGN PATTERN?



Kenali Masalah



Pahami Konteks



Pilih Pola yang Tepat



Terapkan



Evaluasi & Perbaiki



STRATEGY PATTERN

Ganti algoritma/aturan tanpa ubah object utama

KONSEP

Loan (Context)



Object utama
(Tidak berubah)

Strategy



Algoritma/aturan
(Dapat diganti)

Strategy Pattern memungkinkan mengubah algoritma perhitungan (bunga, fee, denda) tanpa mengubah class Loan (object utama).

✓ Mengikuti prinsip: Open/Closed Principle (OCP)

KASUS DI P2P LENDING

Perhitungan yang sering berubah aturannya:



Perhitungan Bunga
(flat, efektif, anuitas)



Fee Platform



Denda Keterlambatan

Perhitungan Pinjaman	
Pokok Pinjaman	Rp 100.000.000
Tenor	12 bulan
Bunga	Rp 12.000.000
Fee Platform	Rp 1.000.000
Denda	Rp 500.000
Total	Rp 113.500.000



CONTOH SCENARIO PENGGUNAAN



Pinjaman UMKM

menggunakan

FlatInterest
(Bunga Flat)

Bunga tetap setiap periode
Mudah dipahami debitur



Pinjaman Konsumtif

menggunakan

EffectiveInterest
(Bunga Efektif)

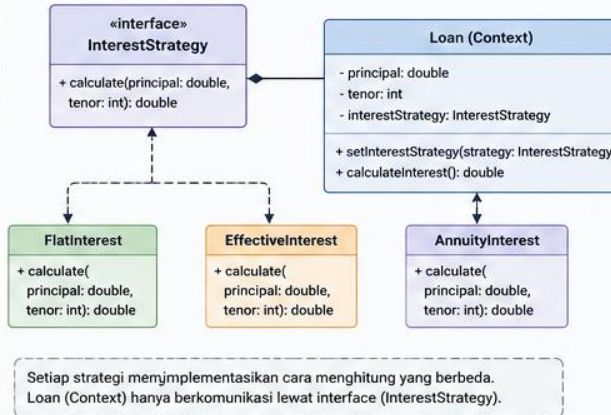
Perhitungan bunga
lebih akurat sesuai
aturan efektif



INSIGHT DOMAIN

- ✓ Cocok untuk produk yang sering berubah rule perhitungan.
- ✓ Perbedaan produk = perbedaan strategi perhitungan.
- ✓ Wajib di Domain Layer, karena ini Core Business Rule.

DIAGRAM KELAS (KONSEP)



CONTOH KODE (JAVA)

```
// Strategy (Interface)
interface InterestStrategy {
    double calculate(double principal, int tenor);
}

// Concrete Strategy
class FlatInterest implements InterestStrategy {
    @Override
    public double calculate(double principal, int tenor) {
        double ratePerPeriod = 0.01; // contoh 1% per periode
        return principal * ratePerPeriod * tenor;
    }
}

class EffectiveInterest implements InterestStrategy {
    @Override
    public double calculate(double principal, int tenor) {
        double annualRate = 0.12; // 12% efektif per tahun
        double monthlyRate = Math.pow(1 + annualRate, 1.0/12) - 1;
        return principal * monthlyRate * tenor;
    }
}

// Context
class Loan {
    private double principal;
    private int tenor;
    private InterestStrategy interestStrategy;

    public void setInterestStrategy(InterestStrategy strategy) {
        this.interestStrategy = strategy;
    }

    public double calculateInterest() {
        return interestStrategy.calculate(principal, tenor);
    }
}

// Penggunaan
Loan loan = new Loan();
loan.setInterestStrategy(new FlatInterest());
double interest = loan.calculateInterest();
```

IMPLEMENTASI DI P2P LENDING

1 Pilih Strategi sesuai produk



UMKM

FlatInterest



Konsumtif

EffectiveInterest

2 Loan menggunakan strategi terpilih



3 Hitung komponen



Bunga

- Flat / Efektif / Anuitas



Fee Platform

- Persentase / Tetap dari Pokok



Denda Keterlambatan

- Persentase per hari / Maksimum

4 Hasil perhitungan

Pokok Pinjaman	Rp 100.000.000
Tenor	12 bulan
Bunga	Rp 12.000.000
Fee Platform	Rp 1.000.000
Denda	Rp 500.000
Total	Rp 113.500.000



KENAPA STRATEGY PATTERN?

- ✓ Ganti rule tanpa ubah core object (Loan)
- ✓ Mudah tambah jenis perhitungan baru
- ✓ Mengurangi if-else/switch yang kompleks
- ✓ Mendukung perubahan bisnis dengan cepat

KAPAN DIPAKAI?

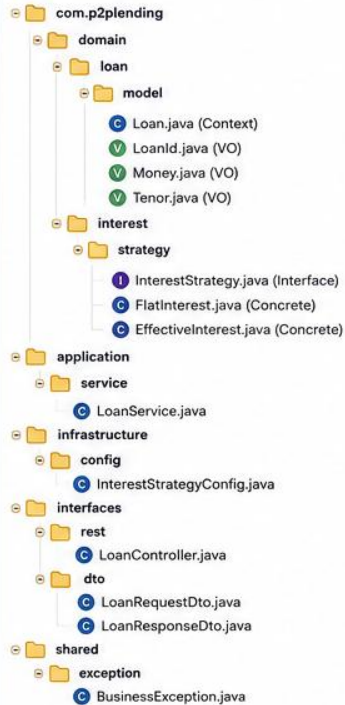
Saat ada banyak variasi algoritma/aturan perhitungan yang bisa berubah-ubah.



STRUKTUR PACKAGE & KELAS (DDD)

STRATEGY PATTERN – P2P LENDING

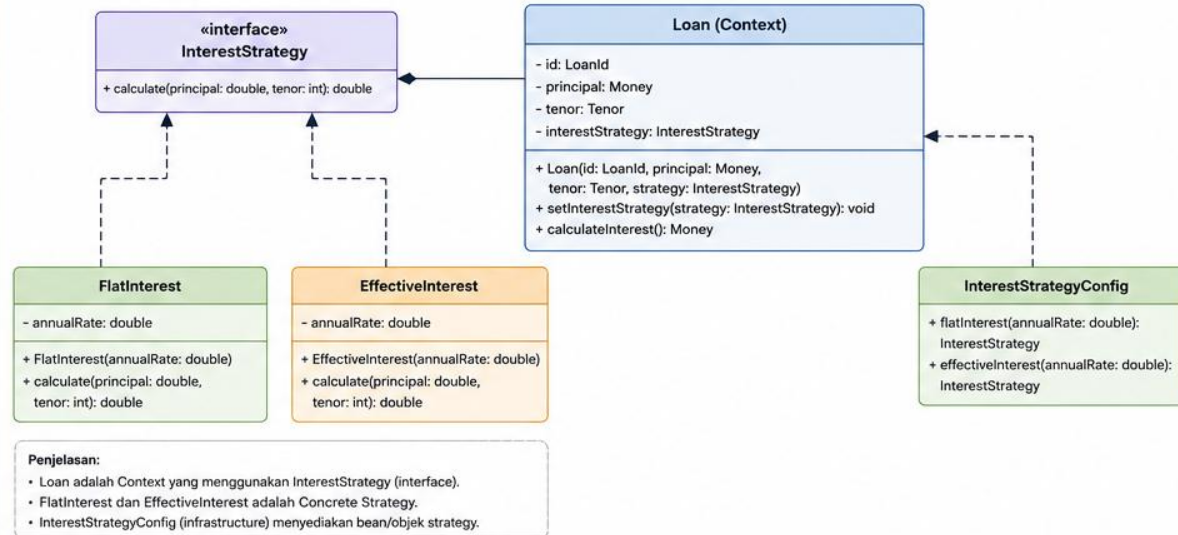
STRUKTUR PACKAGE (DDD LAYERED)



RINGKASAN DDD MAPPING

Layer	Tanggung Jawab
Domain	Core business rule (Strategy, Loan, VO)
Application	Use case / orchestration
Infrastructure	Provide implementation (config, external)
Interfaces	API/Controller, DTO
Shared	Exception, utils, konstanta

DIAGRAM KELAS (KONSEP)



IMPLEMENTASI KELAS (RINGKAS)

Interest/strategy/InterestStrategy.java	Interest/strategy/FlatInterest.java	Interest/strategy/EffectiveInterest.java	loan/model/Loan.java (Context)	infrastructure/config/InterestStrategyConfig.java
<pre> package com.p2plending.domain.interest.strategy; public interface InterestStrategy { double calculate(double principal, int tenor); } </pre>	<pre> package com.p2plending.domain.interest.strategy; public class FlatInterest implements InterestStrategy { private final double annualRate; public FlatInterest(double annualRate) { this.annualRate = annualRate; } @Override public double calculate(double principal, int tenor) { double ratePerPeriod = annualRate / 12.0; return principal * ratePerPeriod * tenor; } } </pre>	<pre> package com.p2plending.domain.interest.strategy; public class EffectiveInterest implements InterestStrategy { private final double annualRate; public EffectiveInterest(double annualRate) { this.annualRate = annualRate; } @Override public double calculate(double principal, int tenor) { double monthlyRate = Math.pow(1 + annualRate, 1.0/12)-1; return principal * monthlyRate * tenor; } } </pre>	<pre> package com.p2plending.domain.loan.model; import com.p2plending.domain.interest.strategy.interest.strategy; public class Loan { private final LoanId id; private final Money principal; private final Tenor tenor; private InterestStrategy interestStrategy; public Loan(LoanId id, Money principal, Tenor tenor, InterestStrategy strategy) { this.id = id; this.principal = principal; this.tenor = tenor; this.interestStrategy = strategy; } public void setInterestStrategy(InterestStrategy strategy) { this.interestStrategy = strategy; } public Money calculateInterest() { double amount = interestStrategy.calculate(principal.getAmount(), tenor.getValue()); return Money.of(amount, principal.getCurrency()); } } </pre>	<pre> package com.p2plending.infrastructure.config; import com.p2plending.domain.interest.strategy.*; public class InterestStrategyConfig { public InterestStrategy flatInterest(double annualRate) { return new FlatInterest(annualRate); } public InterestStrategy effectiveInterest(double annualRate) { return new EffectiveInterest(annualRate); } } </pre>

ALUR PENGGUNAAN



INSIGHT DOMAIN

- ✓ Strategy Pattern cocok untuk rule perhitungan yang sering berubah.
- ✓ Diposisikan di Domain Layer karena termasuk Core Business Rule.
- ✓ Loan (Aggregate Root) tetap stabil, algoritma dapat diganti dengan mudah.



Dari if-else ke Strategy Pattern

Solusi Elegan dengan DDD Mindset

1. TANPA STRATEGY PATTERN (Masalah)

Ubiquitous Language (UL)

Loan Interest Interest Type: FLAT, EFFECTIVE Tenor Principal

Source Code (Tanpa Strategy)

```
class Loan {
    private double principal;
    private int tenor;
    private String interestType; // "FLAT" / "EFFECTIVE"

    double calculateInterest() {
        if (interestType.equals("FLAT")) {
            return principal * 0.1 * tenor;
        }
        if (interestType.equals("EFFECTIVE")) {
            double interest = 0;
            double remaining = principal;
            for (int i = 0; i < tenor; i++) {
                interest += remaining * 0.1;
                remaining -= principal / tenor;
            }
            return interest;
        }
        throw new RuntimeException("Unknown interest type");
    }
}
```

Masalah yang Muncul



1. Pelanggaran Open/Closed Principle

Tambah ANNUITY, PROMO INTEREST, harus ubah class Loan.



2. Domain Knowledge "Terkubur"

Istilah bisnis (Interest) tidak eksplisit, hanya jadi if-else. UL tidak tercermin.



3. Sulit Diuji & Dikembangkan

Logic bercampur dalam Loan, tidak modular.



4. Loan Jadi "Gemuk" (God Object)

Pegang terlalu banyak aturan dan tanggung jawab.

2. DENGAN STRATEGY PATTERN (Solusi)

Ide Strategy Pattern (Dengan DDD Mindset)

- Ubah cara berpikir:

Variasi Perhitungan Bunga

Jadikan Strategy

- Maka:

InterestCalculation sebagai abstraction

- Setiap variasi = implementasi berbeda

- Ubiquitous Language diperjelas

- Bukan lagi:

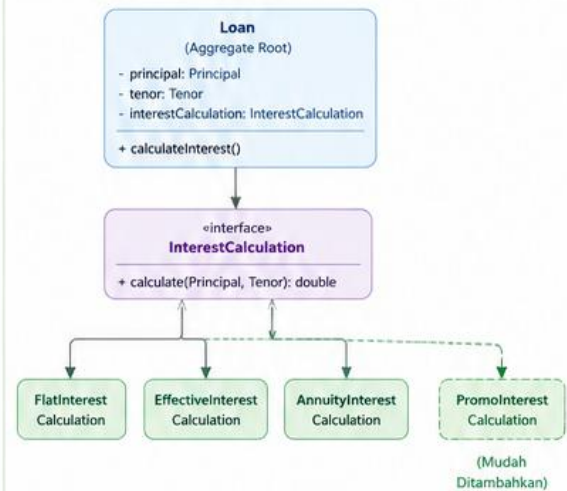
interestType: String

- Tapi:

InterestCalculation

✓ Ini penting dalam DDD!

Arsitektur (Dengan Strategy Pattern)



✓ Open for extension, closed for modification



3. SOURCE CODE (DENGAN STRATEGY PATTERN)

Step 1: Domain Abstraction (Strategy Interface)

```
interface InterestCalculation {
    double calculate(Principal principal, Tenor tenor);
}
```

Step 2: Value Objects (UL yang Kuat)

```
class Principal {
    private double amount;

    Principal(double amount) {
        this.amount = amount;
    }

    double getAmount() {
        return amount;
    }
}
```

```
class Tenor {
    private int months;

    Tenor(int months) {
        this.months = months;
    }

    int getMonths() {
        return months;
    }
}
```

✓ Bukan double, tapi *Principal*.
Nama class = Bahasa Bisnis.

Step 3: Concrete Strategies (Implementasi Berbeda)

```
class FlatInterestCalculation
    implements InterestCalculation {
    public double calculate(Principal p, Tenor t) {
        return p.getAmount() * 0.1 * t.getMonths();
    }
}
```

```
class EffectiveInterestCalculation
    implements InterestCalculation {
    public double calculate(Principal p, Tenor t) {
        double interest = 0;
        double remaining = p.getAmount();
        for (int i = 0; i < t.getMonths(); i++) {
            interest += remaining * 0.1;
            remaining -= p.getAmount() / t.getMonths();
        }
        return interest;
    }
}
```

```
class AnnuityInterestCalculation
    implements InterestCalculation {
    public double calculate(Principal p, Tenor t) {
        double r = 0.1;
        int n = t.getMonths();
        double pv = p.getAmount();
        double annuity = (pv * r) / (1 - Math.pow(1 + r, -n));
        return (annuity * n) - pv; // total interest
    }
}
```

```
class PromoInterestCalculation
    implements InterestCalculation {
    public double calculate(Principal p, Tenor t) {
        double r = 0.05; // promo rate 5%
        return p.getAmount() * r * t.getMonths();
    }
}
```

Mudah menambah variasi tanpa ubah class Loan!

Step 4: Loan sebagai Aggregate Root

```
class Loan {
    private Principal principal;
    private Tenor tenor;
    private InterestCalculation interestCalculation;

    Loan(Principal principal, Tenor tenor,
        InterestCalculation interestCalculation) {
        this.principal = principal;
        this.tenor = tenor;
        this.interestCalculation = interestCalculation;
    }

    double calculateInterest() {
        return interestCalculation.calculate(principal, tenor);
    }
}
```

Step 5: Factory Inject Strategy (DDD Style)

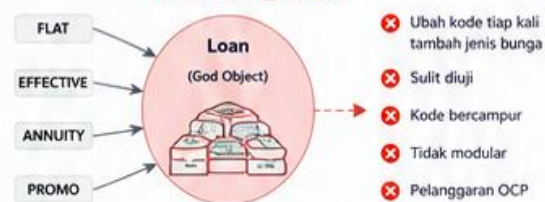
```
class LoanFactory {
    static Loan createFlatLoan(double amount, int months) {
        return new Loan(new Principal(amount),
            new Tenor(months),
            new FlatInterestCalculation());
    }

    static Loan createEffectiveLoan(double amount, int months) {
        return new Loan(new Principal(amount),
            new Tenor(months),
            new EffectiveInterestCalculation());
    }

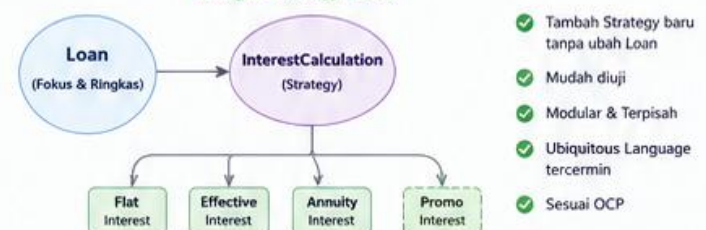
    static Loan createPromoLoan(double amount, int months) {
        return new Loan(new Principal(amount),
            new Tenor(months),
            new PromoInterestCalculation());
    }
}
```

4. PERBANDINGAN VISUAL

Tanpa Strategy (Buruk)



Dengan Strategy (Baik)



5. MANFAAT STRATEGY PATTERN DALAM DDD



Melindungi model domain dari perubahan



Bahasa bisnis (UL) menjadi nyata di kode



Mudah dikembangkan tanpa risiko tinggi



Kode lebih bersih, terfokus & teruji

2. STATE PATTERN

Behavior berubah sesuai state object

TUJUAN



Memungkinkan sebuah object mengubah perilakunya ketika state internalnya berubah, seolah-olah object tersebut berganti class.

KASUS DI P2P LENDING

Lifecycle Loan



BEHAVIOR BERUBAH SESUAI STATE

	Active	Pembayaran normal diterima
	Late	Pembayaran dikenakan penalty
	Default	Tidak bisa bayar normal, hanya settlement
	Closed	Loan selesai, tidak ada transaksi lagi



INSIGHT DOMAIN

State Pattern cocok untuk mengelola lifecycle loan yang kompleks dengan aturan dan behavior yang berbeda di setiap state.

Contoh Business Rule

- Jika Late → kena penalty
- Jika Default → tidak bisa bayar normal
- Beberapa aksi hanya boleh di state tertentu

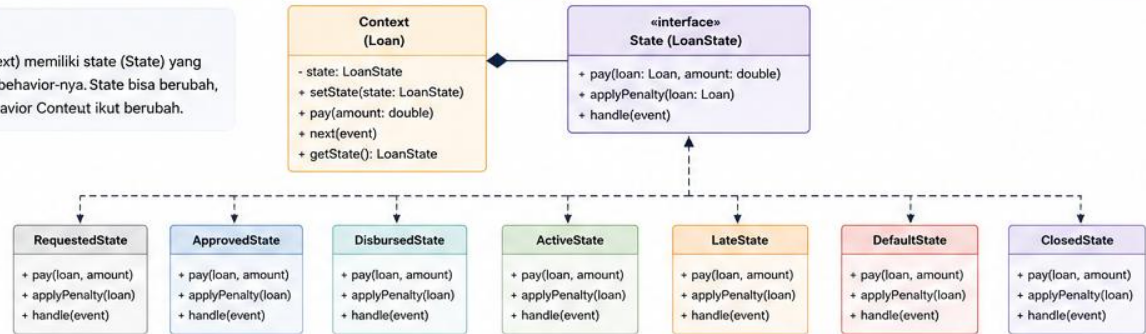
Keuntungan

- ✓ Kode lebih bersih (hindari if-else berantai)
- ✓ Mudah menambah state baru
- ✓ Encapsulation rule per state

A. CONCEPTUAL DIAGRAM (GoF STATE PATTERN)

KONSEP

Object (Context) memiliki state (State) yang menentukan behavior-nya. State bisa berubah, sehingga behavior Context ikut berubah.



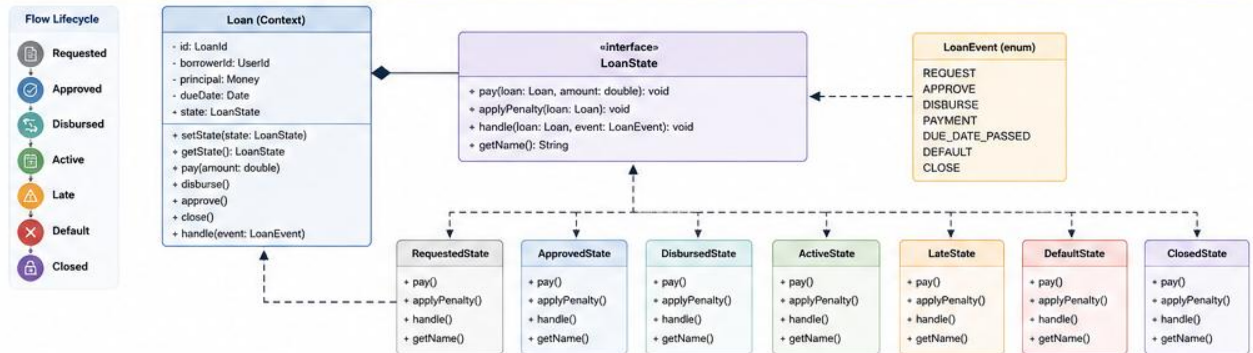
Penjelasan:

- Loan (Context) tidak tahu detail aturan di setiap state.
- Setiap state mengimplementasikan behavior-nya sendiri.
- State dapat berpindah berdasarkan event tertentu.

Kapan digunakan?

- Banyak kondisi / status yang mempengaruhi behavior object
- Behavior berbeda di setiap status
- Harus menghindari banyak if-else atau switch-case

B. DIAGRAM KELAS PENERAPAN – P2P LENDING LOAN LIFECYCLE



Contoh Perilaku per State

State	Perilaku Utama
Requested	Belum disetujui, tidak bisa bayar
Approved	Disetujui, menunggu pencairan
Disbursed	Dana dicairkan, siap aktif
Active	Pembayaran normal diterima
Late	Terlambat, dikenakan penalty
Default	Gagal bayar, hanya settlement
Closed	Loan selesai, semua akses terkunci

Contoh Transisi (Trigger Event)

Dari	Event	Ke
Requested	APPROVE	Approved
Approved	DISBURSE	Disbursed
Disbursed	PAYMENT	Active
Active	DUE_DATE_PASSED	Late
Late	PAYMENT	Active
Late	DEFAULT	Default
Default	SETTLEMENT	Closed
Active/Late	CLOSE	Closed

Contoh Behavior di Beberapa State

ActiveState.pay(loan, amount)

- ✓ Validasi amount
- ✓ Kurangi outstanding
- ✓ Jika lunas → loan.setState(ClosedState)

LateState.pay(loan, amount)

- ✓ Hitung penalty
- ✓ Tambah penalty ke outstanding
- ✓ Terima pembayaran

DefaultState.pay(loan, amount)

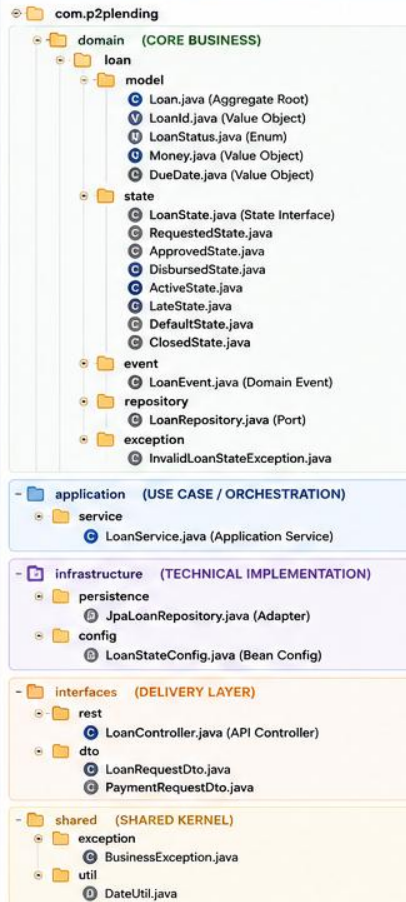
- ✗ Tolak pembayaran normal
- ✓ Hanya terima settlement sesuai rule



STRUKTUR PACKAGE & PENJELASAN PER LAYER – STATE PATTERN (P2P LENDING)

Implementasi DDD (Domain Driven Design)

STRUKTUR PACKAGE



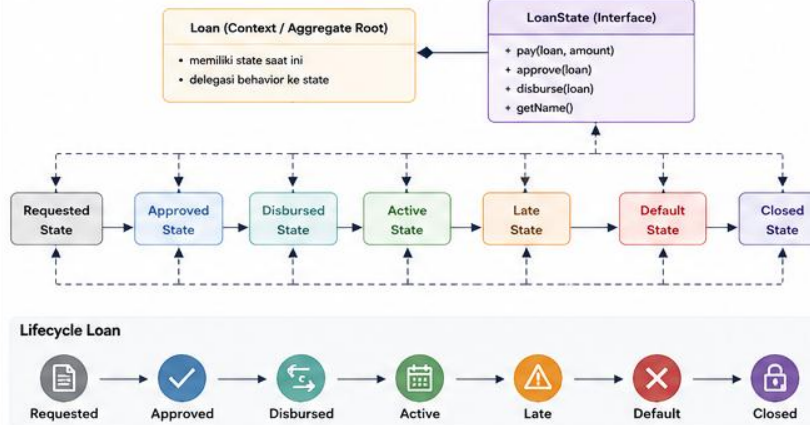
KETERANGAN IKON



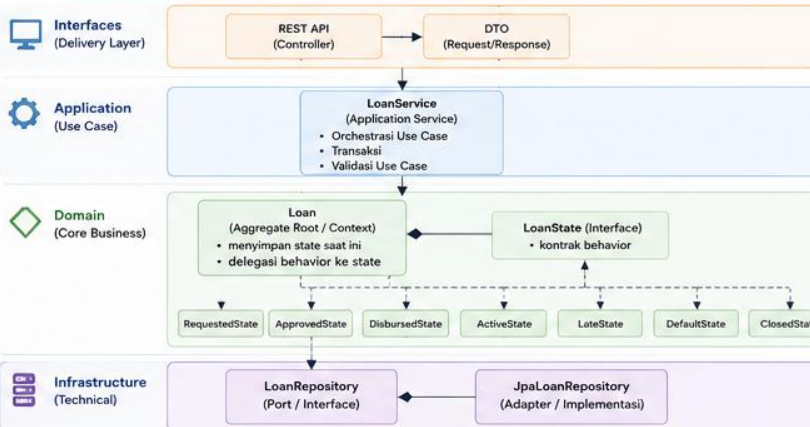
INSIGHT DOMAIN

- State Pattern memastikan behavior berubah sesuai lifecycle loan.
- Contoh aturan bisnis:
 - Late → pembayaran dikenakan penalty
 - Default → tidak bisa bayar normal
- Domain layer tetap bersih, fokus pada aturan bisnis.

DOMAIN MODEL (KONSEP STATE PATTERN)



POSISI SETIAP PACKAGE DALAM ARSITEKTUR



PENJELASAN PER LAYER



1. INTERFACES (DELIVERY LAYER)

- Berinteraksi dengan user / external system (REST API)
- Validasi request, mapping ke DTO
- Tidak berisi business rule

Contoh Komponen

- LoanController
- LoanRequestDto, PaymentRequestDto



2. APPLICATION (USE CASE LAYER)

- Orkestrasi alur use case
- Mengkoordinasikan object domain
- Mengatur transaksi
- Tidak berisi business rule (thin layer)

Contoh Komponen

- LoanService

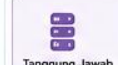


3. DOMAIN (CORE BUSINESS LAYER)

- Inti dari bisnis P2P Lending
- Berisi aturan, lifecycle, state transition
- Menjaga konsistensi & invariant bisnis
- State Pattern diimplementasikan di layer ini

Contoh Komponen

- Loan (Aggregate Root)
- LoanState (Interface)
- Concrete States (RequestedState, ActiveState, etc.)
- LoanRepository (Port)
- Domain Event, Value Object, Exception



4. INFRASTRUCTURE (TECHNICAL LAYER)

- Implementasi teknis (Database, JPA, Config, External Service)
- Adapter dari Port (Repository)
- Dapat diganti tanpa mengubah Domain

Contoh Komponen

- JpaLoanRepository
- LoanStateConfig



5. SHARED (SHARED KERNEL)

- Berisi komponen yang digunakan lintas layer
- Tidak spesifik ke satu domain

Contoh Komponen

- BusinessException
- DateUtil, Constants, Helper

KEUNTUNGAN

- ✓ Mudah menambah state baru tanpa ubah code lama (OCP)
- ✓ Kode lebih maintainable (hindari if-else kompleks)
- ✓ Transisi & aturan state terpusat di Domain
- ✓ Mudah diuji (unit test per state)

CONTOH TRANSISI (SEBAGAI GAMBAR)



3. FACTORY PATTERN

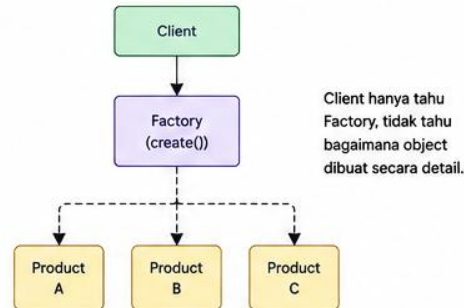
Tujuan: membuat object tanpa expose detail konstruksi

1. PEMAHAMAN KONSEP

Factory Pattern adalah **creational pattern** yang menyediakan interface untuk membuat object, tetapi membiarkan subclass menentukan class object yang akan diinstansiasi.

Tujuan Utama:

- ✓ Membuat object tanpa expose detail konstruksi
- ✓ Memisahkan logika pembuatan object dari penggunaan object
- ✓ Mudah menambah jenis object baru tanpa mengubah kode client (Open/Closed Principle)
- ✓ Menjaga konsistensi dan validasi saat object dibuat

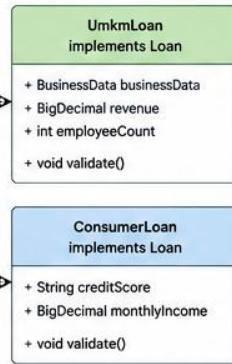


3. CONTOH IMPLEMENTASI (LOAN FACTORY)

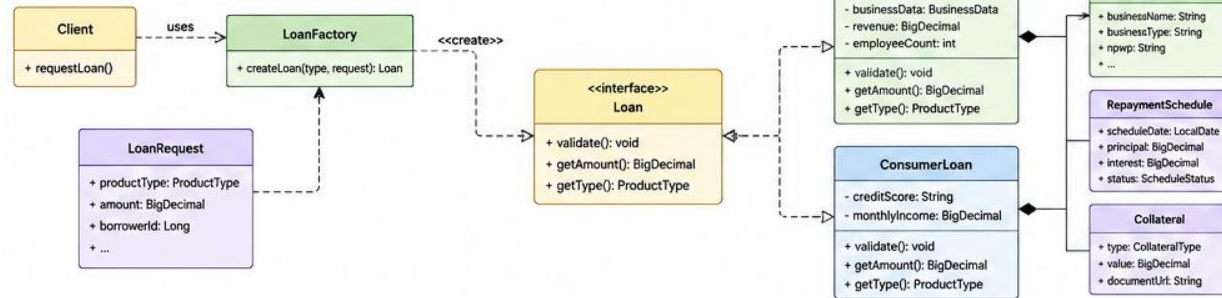
```
enum ProductType {
    UMKM, CONSUMER
}

interface Loan {
    void validate();
}

class LoanFactory {
    public static Loan createLoan(ProductType type, LoanRequest request) {
        if (type == ProductType.UMKM) {
            return new UmkmLoan(request);
        } else if (type == ProductType.CONSUMER) {
            return new ConsumerLoan(request);
        } else {
            throw new IllegalArgumentException("Unknown product type");
        }
    }
}
```



5. DIAGRAM KELAS



2. KASUS DI P2P LENDING

Dalam platform P2P Lending terdapat banyak jenis produk/entitas:



LoanProduct

Berbagai jenis pinjaman

- UMKM Loan
- Consumer Loan
- Pendidikan Loan
- dll



Account

Berbagai jenis akun

- Lender Account
- Borrower Account
- Admin Account
- dll



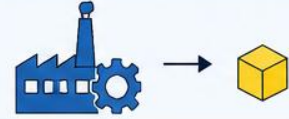
Contract

Berbagai jenis kontrak

- Loan Contract
- Guarantee Contract
- Service Agreement
- dll

Mengapa perlu Factory?

- ✓ Setiap jenis produk/entitas memiliki aturan dan data yang berbeda
- ✓ Validasi awal harus dilakukan sebelum object benar-benar dibuat
- ✓ Menghindari duplikasi kode pembuatan object di banyak tempat
- ✓ Mudah menambah jenis baru tanpa mengubah kode yang sudah ada



4. INSIGHT DOMAIN

Factory digunakan untuk:

- ✓ Enforce rule saat create
- ✓ Validasi awal sebelum object digunakan
- ✓ Menjaga konsistensi data sesuai aturan bisnis

Contoh Aturan Domain

UMKM Loan

WAJIB:

- Punya Business Data
- Punya Revenue
- Punya Jumlah Karyawan

Jika tidak lengkap → Gagal dibuat

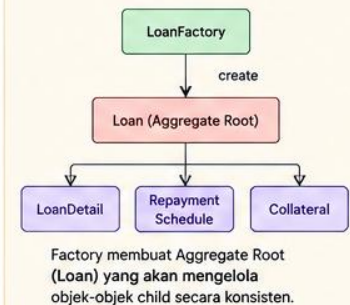
Consumer Loan

WAJIB:

- Punya Credit Score
- Punya Monthly Income

Jika tidak lengkap → Gagal dibuat

Biasanya dipakai bersama
Aggregate Root



Factory membuat Aggregate Root (Loan) yang akan mengelola objek-objek child secara konsisten.

6. KEUNTUNGAN FACTORY PATTERN DI P2P LENDING

- ✓ **Validasi & Enforce Rule**
Semua aturan bisnis penting dicek di satu tempat saat object dibuat.
- ✓ **Konsistensi Object**
Setiap object yang dibuat selalu dalam keadaan valid.
- ✓ **Mudah Maintenance**
Penambahan jenis produk baru cukup di Factory, tidak mengubah client.
- ✓ **Loose Coupling**
Client tidak bergantung pada class konkret, hanya pada interface.
- ✓ **Mendukung Aggregate Root**
Factory memastikan Aggregate Root dibuat dengan data yang konsisten sesuai aturan domain.



3. FACTORY PATTERN

Tujuan: membuat object tanpa expose detail konstruksi

Konsep Singkat

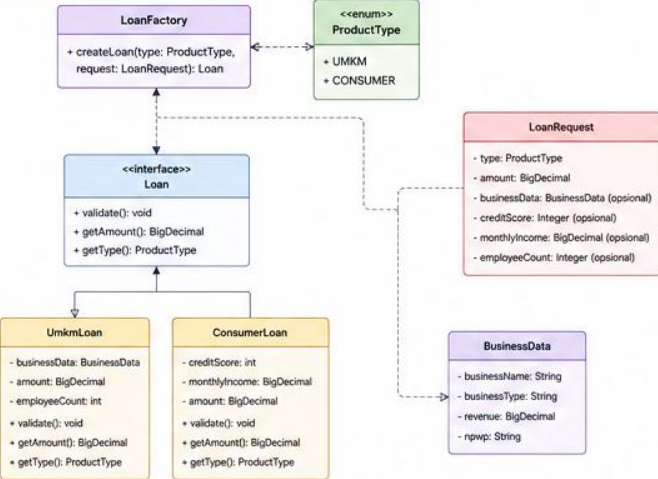
Factory Pattern menyediakan method untuk membuat object dari beberapa class turunan tanpa mengekspos detail konstruksi ke client. Cocok digunakan ketika pembuatan object memiliki aturan/validasi khusus.



1. STRUKTUR PACKAGE



2. DIAGRAM KELAS (UML)



4.4 ConsumerLoan.java

```
package com.example.p2plending.domain.loan;

import java.math.BigDecimal;

public class ConsumerLoan implements Loan {
    private final int creditScore;
    private final BigDecimal monthlyIncome;
    private final BigDecimal amount;

    public ConsumerLoan(int creditScore, BigDecimal monthlyIncome, BigDecimal amount) {
        this.creditScore = creditScore;
        this.monthlyIncome = monthlyIncome;
        this.amount = amount;
    }

    @Override
    public void validate() {
        if (creditScore < 300) {
            throw new IllegalArgumentException("Credit score minimal 300");
        }
        if (monthlyIncome == null || monthlyIncome.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Monthly income tidak valid");
        }
        if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Amount tidak valid");
        }
    }

    @Override
    public BigDecimal getAmount() { return amount; }

    @Override
    public ProductType getType() { return ProductType.CONSUMER; }

    public int getCreditScore() { return creditScore; }
    public BigDecimal getMonthlyIncome() { return monthlyIncome; }
}
```

4.6 Contoh Penggunaan (DemoApplication.java)

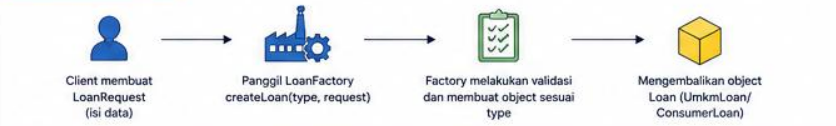
```
package com.example.p2plending.application;

import com.example.p2plending.domain.loan;
import java.math.BigDecimal;

public class DemoApplication {
    public static void main(String[] args) {
        BusinessData businessData = new BusinessData("Toko Maju", "Retail", new BigDecimal("125000000"));
        LoanRequest loanRequest = new LoanRequest();
        loanRequest.setBusinessData(businessData);
        loanRequest.setAmount(new BigDecimal("100000000"));
        loanRequest.setEmployeeCount(10);
        loanRequest.setMonthlyIncome(new BigDecimal("100000000"));

        Loan loan = LoanFactory.createLoan(ProductType.CONSUMER, loanRequest);
        System.out.println("UMKM Loan berhasil dibuat: " + loan.getType() + " - " + loan.getAmount());
    }
}
```

3. ALUR PROGRAM



4. CONTOH PROGRAM LENGKAP

4.1 Enum ProductType.java

```
package com.example.p2plending.domain.loan;

public enum ProductType {
    UMKM,
    CONSUMER
}
```

4.2 Loan.java (Interface)

```
package com.example.p2plending.domain.loan;

import java.math.BigDecimal;

public interface Loan {
    void validate();
    BigDecimal getAmount();
    ProductType getType();
}
```

4.5 LoanFactory.java

```
package com.example.p2plending.domain.loan;

public class LoanFactory {
    public static Loan createLoan(ProductType type, LoanRequest request) {
        if (type == ProductType.UMKM) {
            if (request.getBusinessData() == null) {
                throw new IllegalArgumentException("Business data wajib diisi untuk UMKM Loan");
            }
            if (request.getEmployeeCount() == null || request.getEmployeeCount() <= 0) {
                throw new IllegalArgumentException("Employee count wajib diisi untuk UMKM Loan");
            }
            Loan loan = new UmkmLoan(request.getBusinessData(), request.getAmount(), request.getEmployeeCount());
            loan.validate();
            return loan;
        } else if (type == ProductType.CONSUMER) {
            if (request.getCreditScore() == null || request.getCreditScore() < 300) {
                throw new IllegalArgumentException("Credit score minimal 300 untuk Consumer Loan");
            }
            if (request.getMonthlyIncome() == null || request.getMonthlyIncome().compareTo(BigDecimal.ZERO) <= 0) {
                throw new IllegalArgumentException("Monthly income wajib diisi untuk Consumer Loan");
            }
            Loan loan = new ConsumerLoan(request.getCreditScore(), request.getMonthlyIncome(), request.getAmount());
            loan.validate();
            return loan;
        }
        throw new IllegalArgumentException("Type produk tidak didukung");
    }
}
```

4.7 Kelas Pendukung (LoanRequest & BusinessData)

LoanRequest.java

```
package com.example.p2plending.domain.loan;

import java.math.BigDecimal;

public class LoanRequest {
    private ProductType type;
    private BigDecimal amount;
    private BusinessData businessData;
    private Integer employeeCount;
    private BigDecimal monthlyIncome;

    // getter & setter
    public ProductType getType() { return type; }
    public void setType(ProductType type) { this.type = type; }
    public BigDecimal getAmount() { return amount; }
    public void setAmount(BigDecimal amount) { this.amount = amount; }
    public BusinessData getBusinessData() { return businessData; }
    public void setBusinessData(BusinessData businessData) { this.businessData = businessData; }
    public Integer getEmployeeCount() { return employeeCount; }
    public void setEmployeeCount(Integer employeeCount) { this.employeeCount = employeeCount; }
    public BigDecimal getMonthlyIncome() { return monthlyIncome; }
    public void setMonthlyIncome(BigDecimal monthlyIncome) { this.monthlyIncome = monthlyIncome; }
}
```

BusinessData.java

```
package com.example.p2plending.domain.loan;

import java.math.BigDecimal;

public class BusinessData {
    private String businessName;
    private String businessType;
    private BigDecimal revenue;
    private String npwp;

    // getter & setter
    public String getBusinessName() { return businessName; }
    public void setBusinessName(String businessName) { this.businessName = businessName; }
    public String getBusinessType() { return businessType; }
    public void setBusinessType(String businessType) { this.businessType = businessType; }
    public BigDecimal getRevenue() { return revenue; }
    public void setRevenue(BigDecimal revenue) { this.revenue = revenue; }
    public String getNpwp() { return npwp; }
    public void setNpwp(String npwp) { this.npwp = npwp; }
}
```

4.3 UmkmLoan.java

```
package com.example.p2plending.domain.loan;

import java.math.BigDecimal;

public class UmkmLoan implements Loan {
    private final BusinessData businessData;
    private final BigDecimal amount;
    private final int employeeCount;

    public UmkmLoan(BusinessData businessData, BigDecimal amount, int employeeCount) {
        this.businessData = businessData;
        this.amount = amount;
        this.employeeCount = employeeCount;
    }

    @Override
    public void validate() {
        if (businessData == null) {
            throw new IllegalArgumentException("UMKM Loan wajib memiliki Business Data");
        }
        if (employeeCount <= 0) {
            throw new IllegalArgumentException("Employee count harus lebih dari 0");
        }
        if (amount == null || amount.compareTo(BigDecimal.ZERO) <= 0) {
            throw new IllegalArgumentException("Amount tidak valid");
        }
    }

    @Override
    public BigDecimal getAmount() { return amount; }

    @Override
    public ProductType getType() { return ProductType.UMKM; }

    public BusinessData getBusinessData() { return businessData; }
    public int getEmployeeCount() { return employeeCount; }
}
```

5. OUTPUT PROGRAM (Contoh)

```
UMKM Loan berhasil dibuat: UMKM - 100000000
Consumer Loan berhasil dibuat: CONSUMER - 150000000
```

6. PEMAHAMAN INTI

- ✓ Factory menyembunyikan proses pembuatan object dan validasi awal
- ✓ Client hanya perlu tahu interface (Loan), tidak perlu tahu class konkretnya
- ✓ Mudah menambah jenis produk baru tanpa mengubah client (OCP)
- ✓ Validasi khusus tiap produk ditempatkan di dalam Factory/Constructor (enforce rule)
- ✓ Sering digunakan bersama Aggregate Root pada DDD



FACTORY PATTERN DI P2P LENDING

Membuat Object dengan Validasi & Invariant di Domain

Tujuan
Membuat object tanpa expose detail konstruksi, dengan validasi invariants.



1. PROBLEM (TANPA FACTORY PATTERN)

Bayangkan kita punya beberapa jenis loan:

- UMKM Loan
- Consumer Loan

MASALAH YANG MUNCUL

- Logic domain tersebar**
Validasi domain ada di LoanService Bukan di domain model
melanggar prinsip rich domain model
- Sulit dikembangkan**
Tambah produk baru: harus ubah LoanService rawan bug
- Tidak ada jaminan invariant**
Kalau developer lain langsung pakai:
`new UmkmLoan(...)`
bisa bypass validasi!

SOURCE CODE (TANPA FACTORY)

```
class LoanService {
    Loan createLoan(String type, double amount,
        int tenor, Map<String, Object> data) {

        if (type.equals("UMKM")) {
            if (!data.containsKey("businessName")) {
                throw new RuntimeException(
                    "UMKM loan must have business data");
            }
            return new UmkmLoan(amount, tenor, data);
        }

        if (type.equals("CONSUMER")) {
            if (!data.containsKey("creditScore")) {
                throw new RuntimeException(
                    "Consumer loan must have credit score");
            }
            return new ConsumerLoan(amount, tenor, data);
        }

        throw new RuntimeException("Unknown loan type");
    }
}
```

2. IDE FACTORY PATTERN



3. SOURCE CODE (DENGAN FACTORY PATTERN)

STEP 1: DOMAIN ENTITY (ABSTRACT LOAN)

```
abstract class Loan {
    protected double amount;
    protected int tenor;

    Loan(double amount, int tenor) {
        this.amount = amount;
        this.tenor = tenor;
    }
}
```

STEP 2: CONCRETE LOAN

```
class UmkmLoan extends Loan {
    private String businessName;

    UmkmLoan(double amount, int tenor,
        String businessName) {
        super(amount, tenor);
        this.businessName = businessName;
    }
}

class ConsumerLoan extends Loan {
    private int creditScore;

    ConsumerLoan(double amount, int tenor,
        int creditScore) {
        super(amount, tenor);
        this.creditScore = creditScore;
    }
}
```

STEP 3: FACTORY

```
class LoanFactory {
    static Loan createUmkmLoan(double amount, int tenor, String businessName) {
        if (businessName == null || businessName.isEmpty()) {
            throw new RuntimeException("Business name is required");
        }
        if (amount < 1_000_000) {
            throw new RuntimeException("Minimum UMKM loan is 1,000,000");
        }
        return new UmkmLoan(amount, tenor, businessName);
    }

    static Loan createConsumerLoan(double amount, int tenor, int creditScore) {
        if (creditScore < 600) {
            throw new RuntimeException("Credit score too low");
        }
        return new ConsumerLoan(amount, tenor, creditScore);
    }
}
```

STEP 4: SERVICE JADI BERSIH

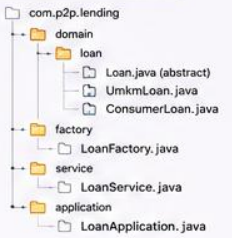
```
class LoanService {
    Loan createUmkmLoan(double amount,
        int tenor, String businessName) {
        return LoanFactory.createUmkmLoan(
            amount, tenor, businessName);
    }

    Loan createConsumerLoan(double amount,
        int tenor, int creditScore) {
        return LoanFactory.createConsumerLoan(
            amount, tenor, creditScore);
    }
}
```

4. ALUR PEMBUATAN LOAN (DENGAN FACTORY)



5. STRUKTUR PACKAGE



KETERANGAN:

- domain.loan
Beri entity Loan dan turunannya
- domain.factory
Beri logic pembuatan object dan validasi invariants
- service
Beri use case / orkestrasi
- application
Entry point aplikasi

6. PERBANDINGAN: TANPA FACTORY vs DENGAN FACTORY

	TANPA FACTORY PATTERN	DENGAN FACTORY PATTERN
Tempat Validasi	Di LoanService ❌	Di LoanFactory ✅
Tanggung Jawab	Service terlalu banyak kerjaan	Factory bertanggung jawab membuat object valid
Konsistensi Data	Berisiko tidak konsisten	Terjamin (invariant terjaga)
Extensibility	Sulit tambah produk baru	Mudah tambah produk baru
Pemeliharaan	Susah dipelihara	Mudah dipelihara
Rich Domain Model	Tidak tercapai	Tercapai (domain yang kuat)
Testability	Lebih sulit di-test	Lebih mudah di-test (terisolasi)

7. CONTOH PENGGUNAAN

```
// Di Application / Controller
LoanService loanService = new LoanService();

Loan loan1 = loanService.createUmkmLoan(
    10_000_000, 12, "Toko Makmur");
);

Loan loan2 = loanService.createConsumerLoan(
    5_000_000, 12, 720);
);

System.out.println(loan1.getClass().getSimpleName()); // UmkmLoan
System.out.println(loan2.getClass().getSimpleName()); // ConsumerLoan
```

8. KESIMPULAN

- Gunakan Factory Pattern ketika:**
- ✓ Membuat banyak jenis object dengan aturan/validasi berbeda
 - ✓ Ingin memastikan setiap object selalu valid sejak awal
 - ✓ Ingin memusatkan logic pembuatan object di satu tempat
 - ✓ Ingin memudahkan penambahan produk baru tanpa ubah banyak kode

Ingat: Factory bukan hanya "membuat object", tapi menjaga kualitas data dan konsistensi domain. Ini sangat penting di P2P Lending yang banyak aturan bisnis & risiko finansial.



4. COMPOSITE PATTERN

Tujuan: treat object tunggal dan kelompok secara uniform

Composite Pattern memungkinkan kita memperlakukan object tunggal (single object) dan kumpulan object (composite) secara sama melalui interface yang sama. Client tidak perlu tahu apakah itu 1 item atau sekumpulan item.



1. PEMAHAMAN KONSEP

Inti Konsep

Ada interface yang sama untuk:

- Object tunggal (Leaf)
- Object kelompok (Composite)

Sehingga client dapat memprosesnya secara uniform.

Analogi Sederhana

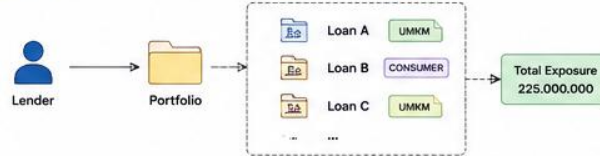
Folder di komputer. Folder bisa berisi file atau folder lain. Kita bisa menghitung total ukuran semua file tanpa perlu tahu apakah itu file tunggal atau kumpulan folder.

Tujuan Utama

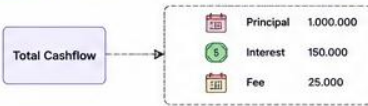
- ✓ Menyederhanakan struktur data hierarki
- ✓ Mempermudah perhitungan agregasi
- ✓ Membuat kode lebih fleksibel dan mudah dikembangkan
- ✓ Client menjadi lebih sederhana

2. KASUS DI P2P LENDING

Contoh 1: Portfolio Lender terdiri dari banyak Loan



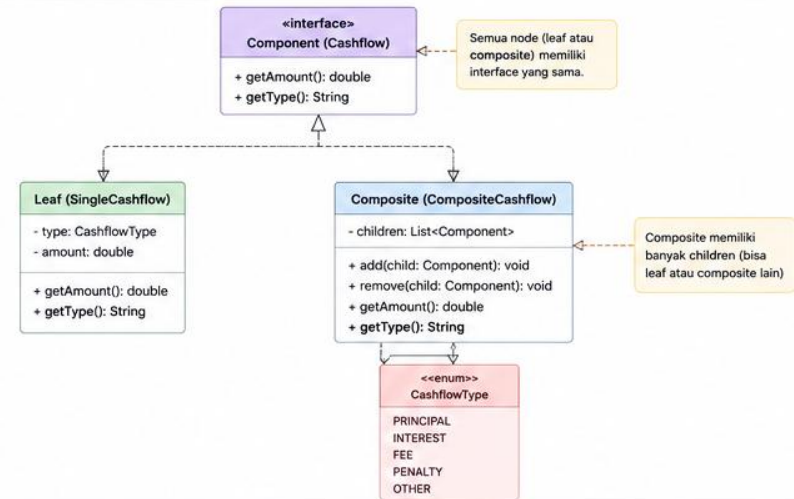
Contoh 2: Cashflow terdiri dari Principal, Interest, Fee



Mengapa Composite Pattern?

- ✓ Struktur seperti tree (hierarki)
- ✓ Perhitungan total/summary menjadi lebih mudah
- ✓ Menambahkan jenis komponen baru lebih fleksibel
- ✓ Sangat cocok untuk laporan keuangan, IFRS 17 grouping, portfolio analysis

3. DIAGRAM KELAS (UML)



4. CONTOH IMPLEMENTASI (JAVA)

```
// 1. Component Interface
interface Cashflow {
    double getAmount();
    String getType();
}

// 2. Leaf - SingleCashflow
class SingleCashflow implements Cashflow {
    private final CashflowType type;
    private final double amount;

    public SingleCashflow(CashflowType type, double amount) {
        this.type = type;
        this.amount = amount;
    }

    @Override public double getAmount() { return amount; }
    @Override public String getType() { return type.name(); }
}

// 3. Composite - CompositeCashflow
class CompositeCashflow implements Cashflow {
    private final List<Cashflow> children = new ArrayList<>();
    private final String name;

    public CompositeCashflow(String name) { this.name = name; }

    public void add(Cashflow cashflow) { children.add(cashflow); }
    public void remove(Cashflow cashflow) { children.remove(cashflow); }

    @Override public double getAmount() {
        return children.stream().mapToDouble(Cashflow::getAmount).sum();
    }

    @Override public String getType() { return "COMPOSITE:" + name; }
}

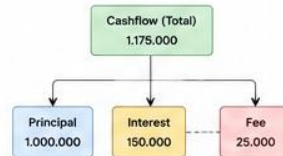
// 4. Penggunaan
Cashflow principal = new SingleCashflow(CashflowType.PRINCIPAL, 1_000_000);
Cashflow interest = new SingleCashflow(CashflowType.INTEREST, 150_000);
Cashflow fee = new SingleCashflow(CashflowType.FEE, 25_000);

Cashflow cashflow = new CompositeCashflow("Cashflow Loan #123");
((CompositeCashflow) cashflow).add(principal);
((CompositeCashflow) cashflow).add(interest);
((CompositeCashflow) cashflow).add(fee);

System.out.println("Total Cashflow = " + cashflow.getAmount()); // 1.175.000
```

Output:
Total Cashflow = 1175000.0

5. DIAGRAM STRUKTUR CONTOH CASHFLOW



Keterangan:
• Kotak hijau: Composite
• Kotak warna: Leaf
• Hitung total dari bawah ke atas

6. INSIGHT DOMAIN

Composite Pattern sangat cocok untuk:

- ✓ Laporan keuangan (Financial Statement, Aging, Cashflow Report)
- ✓ Agregasi nilai finansial dari berbagai level
- ✓ IFRS 17 style grouping (contract groups, coverage units, cashflow aggregation)
- ✓ Portfolio analysis (total exposure, total ECL, total return, dll)

Manfaat

- 📊 Perhitungan total menjadi konsisten dan mudah
- 📁 Struktur data hierarki menjadi lebih fleksibel
- 🔧 Menambah jenis komponen baru tanpa ubah client
- 🧹 Code lebih bersih (client hanya berinteraksi dengan interface)

7. CONTOH PENERAPAN LAIN DI P2P LENDING

Area	Contoh Penerapan
Portfolio Lender	Portfolio (Composite) → terdiri dari banyak Loan (Leaf/Composite)
Cashflow	Total Cashflow (Composite) → Principal, Interest, Fee, Penalty (Leaf)
Payment Schedule	Schedule (Composite) → Installment (Leaf) → Sub Installment (Leaf)
Collateral	Collateral Package (Composite) → Collateral Item (Leaf)
Report	Report (Composite) → Section (Composite) → Line Item (Leaf)

8. KELEBIHAN & KONDISI PENGGUNAAN

Kelebihan

- ✓ Client sederhana (cukup panggil interface)
- ✓ Memudahkan agregasi dan perhitungan total
- ✓ Struktur hierarki dapat berkembang
- ✓ Mendukung reusability dan fleksibilitas tinggi

Kapan Digunakan?

Gunakan Composite Pattern ketika:

- Data berbentuk tree/hierarki
- Ada kebutuhan agregasi/rekap total
- Struktur bisa terdiri dari single item atau group item



COMPOSITE PATTERN

Tujuan: treat object tunggal dan kelompok secara uniform

Composite Pattern memungkinkan kita memperlakukan object tunggal (single object) dan kumpulan object (composite) secara sama melalui interface yang sama.
Client tidak perlu tahu apakah itu 1 item atau sekumpulan item.

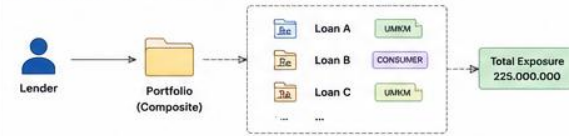


1. STRUKTUR PACKAGE (PROJECT)

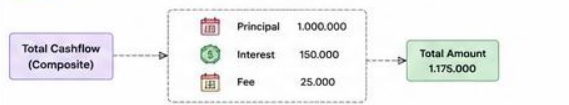


2. KASUS DI P2P LENDING

Contoh 1: Portfolio Lender terdiri dari banyak Loan (Leaf/Composite)



Contoh 2: Cashflow terdiri dari Principal, Interest, Fee (Leaf/Composite)



7. SOURCE CODE LENGKAP

```
Cashflow.java
package com.example.p2plending.cashflow.model;

import com.example.p2plending.common.Money;

public interface Cashflow {
    Money getAmount();
    CashflowType getType();
}
```

```
CashflowType.java
package com.example.p2plending.cashflow.model;

public enum CashflowType {
    PRINCIPAL,
    INTEREST,
    FEE,
    PENALTY,
    OTHER
}
```

```
Money.java
package com.example.p2plending.common;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class Money {
    private final BigDecimal amount;

    public Money(BigDecimal amount) {
        this.amount = amount == null ? BigDecimal.ZERO : amount;
    }

    public static Money of(long value) {
        return new Money(BigDecimal.valueOf(value));
    }

    public Money add(Money other) {
        return new Money(this.amount.add(other.amount));
    }

    public BigDecimal asDecimal() {
        return amount;
    }

    @Override
    public String toString() {
        return amount.setScale(0, RoundingMode.HALF_UP).toString();
    }
}
```

```
CompositePatternDemo.java (Main)
package com.example.p2plending.application;

import com.example.p2plending.cashflow.model.CompositeCashflow;
import com.example.p2plending.cashflow.model.CashflowReportService;
import com.example.p2plending.cashflow.model.Cashflow;
import com.example.p2plending.cashflow.model.SingleCashflow;

public class CompositePatternDemo {
    public static void main(String[] args) {
        CashflowReportService service = new CashflowReportService();

        CompositeCashflow totalCashflow = service.buildCashflowExample();

        Money total = service.getTotalAmount(totalCashflow);
        System.out.println("Total Cashflow = " + total); // 1.175.000
        System.out.println();

        System.out.println("---- Detail Cashflow ----");
        service.printReport(totalCashflow, 0);
    }
}
```

Output (Console)

```
Total Cashflow = 1175000

--- Detail Cashflow ---
- OTHER : 1175000
- PRINCIPAL : 1000000
- INTEREST : 150000
- FEE : 25000
```

```
SingleCashflow.java
package com.example.p2plending.cashflow.model;

import com.example.p2plending.common.Money;

public class SingleCashflow implements Cashflow {
    private final CashflowType type;
    private final Money amount;

    public SingleCashflow(CashflowType type, Money amount) {
        this.type = type;
        this.amount = amount;
    }

    @Override
    public Money getAmount() {
        return amount;
    }

    @Override
    public CashflowType getType() {
        return type;
    }

    @Override
    public String toString() {
        return "SingleCashflow{" +
            "type=" + type +
            ", amount=" + amount +
            '}';
    }
}
```

```
CompositeCashflow.java
package com.example.p2plending.cashflow.model;

import com.example.p2plending.common.Money;
import java.util.ArrayList;
import java.util.List;

public class CompositeCashflow implements Cashflow {
    private final List<Cashflow> children = new ArrayList<>();
    private final String name;

    public CompositeCashflow(String name) {
        this.name = name;
    }

    public void add(Cashflow cashflow) {
        if (cashflow != null) {
            children.add(cashflow);
        }
    }

    public void remove(Cashflow cashflow) {
        children.remove(cashflow);
    }

    public List<Cashflow> getChildren() {
        return children;
    }

    @Override
    public Money getAmount() {
        Money total = Money.of(0);
        for (Cashflow c : children) {
            total = total.add(c.getAmount());
        }
        return total;
    }

    @Override
    public CashflowType getType() {
        return CashflowType.OTHER; // composite type
    }

    @Override
    public String toString() {
        return "CompositeCashflow{" +
            "name=" + name + ", " +
            "children=" + children +
            '}';
    }
}
```

```
CashflowTest.java
package com.example.p2plending.cashflow;

import com.example.p2plending.cashflow.model.*;
import com.example.p2plending.common.Money;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

public class CashflowTest {
    @Test
    public void testCompositeAmount() {
        Cashflow principal = new SingleCashflow(CashflowType.PRINCIPAL, Money.of(1000000));
        Cashflow interest = new SingleCashflow(CashflowType.INTEREST, Money.of(150000));
        Cashflow fee = new SingleCashflow(CashflowType.FEE, Money.of(25000));

        CompositeCashflow total = new CompositeCashflow("Total");
        total.add(principal);
        total.add(interest);
        total.add(fee);

        assertEquals(1175000, total.getAmount().asDecimal().longValue());
    }
}
```

8. KAPAN MENGGUNAKAN COMPOSITE PATTERN?

- ✓ Saat Anda memiliki struktur hierarki (tree) dari object.
Contoh: Portfolio → Loan → Cashflow Item
- ✓ Client perlu melakukan operasi yang sama pada object tunggal maupun kumpulan object.

- ✓ Ingin menyederhanakan kode client (tidak perlu cek apakah itu single atau group).
- ✓ Sering digunakan pada fitur laporan, perhitungan agregasi, dan analisis portfolio.

9. MANFAAT DI P2P LENDING



Perhitungan Total Lebih Konsisten



Struktur Hierarki Lebih Fleksibel



Mudah Dikembangkan (Extensible)



Cocok untuk Laporan dan Clean Kompleks



5. DECORATOR PATTERN

Tujuan: menambahkan behavior tanpa ubah class utama

1. PEMAHAMAN KONSEP

Inti Konsep

Decorator membangun object (component) dengan object lain (decorator) untuk menambahkan behavior tambahan.

- ✓ Tidak mengubah class utama
- ✓ Fleksibel & dinamis (runtime)
- ✓ Bisa stacking banyak decorator
- ✓ Lebih baik daripada inheritance untuk fitur opsional

Analogi Sederhana

Seperti membeli kopi: Kopi (produk dasar) bisa ditambah gula, susu, whipped cream, dll sesuai keinginan pelanggan tanpa mengubah resep kopi aslinya.



Kapan Menggunakan?

- ✓ Ketika ada fitur opsional / tambahan
- ✓ Ketika tidak ingin membuat banyak subclass
- ✓ Ketika kombinasi fitur bisa berubah-ubah
- ✓ Ketika ingin menambah behavior secara dinamis

4. CONTOH KODE LENGKAP (JAVA)

4.1 Interface (Component)

```
public interface Loan {
    double getCost();
    String getDescription();
}
```

4.2 Concrete Component

```
public class SimpleLoan implements Loan {
    private double baseCost;
    private String description;

    public SimpleLoan(double baseCost, String description) {
        this.baseCost = baseCost;
        this.description = description;
    }

    @Override
    public double getCost() {
        return baseCost;
    }

    @Override
    public String getDescription() {
        return description + " (Base Cost: " + baseCost + ")";
    }
}
```

4.3 Insurance Decorator

```
public class InsuranceDecorator extends LoanDecorator {
    private double insuranceFee;

    public InsuranceDecorator(Loan loan, double insuranceFee) {
        super(loan);
        this.insuranceFee = insuranceFee;
    }

    @Override
    public double getCost() {
        return loan.getCost() + insuranceFee;
    }

    @Override
    public String getDescription() {
        return loan.getDescription() + " + Insurance: " + insuranceFee;
    }
}
```

4.3 Abstract Decorator

```
public abstract class LoanDecorator implements Loan {
    protected Loan loan;

    public LoanDecorator(Loan loan) {
        this.loan = loan;
    }
}
```

4.4 Penalty Decorator

```
public class PenaltyDecorator extends LoanDecorator {
    private double penalty;

    public PenaltyDecorator(Loan loan, double penalty) {
        super(loan);
        this.penalty = penalty;
    }

    @Override
    public double getCost() {
        return loan.getCost() + penalty;
    }

    @Override
    public String getDescription() {
        return loan.getDescription() + " + Penalty: " + penalty;
    }
}
```

4.6 Cashback Decorator

```
public class CashbackDecorator extends LoanDecorator {
    private double cashback;

    public CashbackDecorator(Loan loan, double cashback) {
        super(loan);
        this.cashback = cashback;
    }

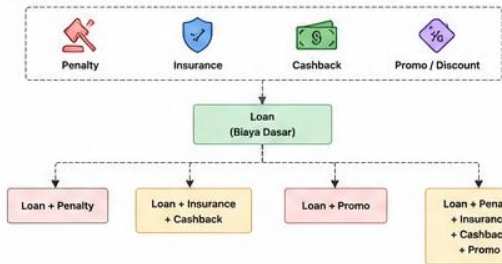
    @Override
    public double getCost() {
        return loan.getCost() - cashback;
    }

    @Override
    public String getDescription() {
        return loan.getDescription() + " + Cashback: " + cashback;
    }
}
```

Decorator Pattern membantu kita memahami responsibility / behavior pada object secara dinamis tanpa mengubah struktur class aslinya. Lebih fleksibel daripada inheritance karena kombinasi fitur bisa dipilih saat runtime.

2. KASUS DI P2P LENDING

Loan memiliki biaya dasar, tetapi bisa memiliki tambahan fitur (opsional):

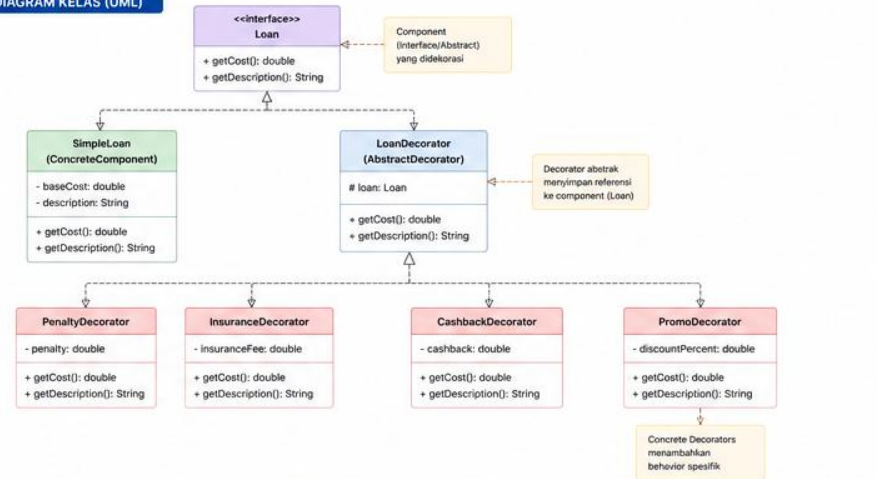


Setiap kombinasi dapat dibuat tanpa membuat subclass baru untuk setiap kemungkinan.

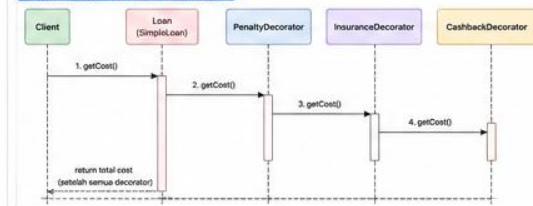
Manfaat di P2P Lending

- ✓ Menambahkan fitur seperti penalty, insurance, cashback, promo tanpa mengubah class Loan
- ✓ Kombinasi fitur lebih fleksibel (bisa berbeda-beda di setiap produk atau user)
- ✓ Mudah dikembangkan (open/closed principle)
- ✓ Cocok untuk policy/biaya tambahan yang dinamis dan opsional

3. DIAGRAM KELAS (UML)



5. ALUR KERJA (SEQUENCE DIAGRAM)



4.8 Client / Penggunaan

```
public class DecoratorPatternDemo {
    public static void main(String[] args) {
        Loan loan = new SimpleLoan(100000000, "Personal Loan");

        System.out.println("==== LOAN TANPA TAMBAHAN ===");
        printLoan(loan);

        System.out.println("==== LOAN + PENALTY ===");
        Loan loanWithPenalty = new PenaltyDecorator(loan, 500000);
        printLoan(loanWithPenalty);

        System.out.println("==== LOAN + INSURANCE + CASHBACK ===");
        Loan loanWithInsuranceCashback = new CashbackDecorator(
            new InsuranceDecorator(loan, 300000),
            200000
        );
        printLoan(loanWithInsuranceCashback);

        System.out.println("==== LOAN + CASHBACK + PROMO ===");
        Loan loanFull = new PromoDecorator(
            new InsuranceDecorator(
                new CashbackDecorator(
                    new SimpleLoan(100000000, "Personal Loan")
                ), 10 // diskon 10%
            ), 10 // diskon 10%
        );
        printLoan(loanFull);

        private static void printLoan(Loan loan) {
            System.out.println("Description: " + loan.getDescription());
            System.out.println("Total Cost : " + loan.getCost());
        }
    }
}
```

Output (Contoh)

```
==== LOAN TANPA TAMBAHAN ===
Description: Personal Loan (Base Cost: 100000000.0)
Total Cost : 100000000.0

==== LOAN + PENALTY ===
Description: Personal Loan (Base Cost: 100000000.0)
Penalty: 500000.0
Total Cost : 105000000.0

==== LOAN + INSURANCE + CASHBACK ===
Description: Personal Loan (Base Cost: 100000000.0)
Insurance: 300000.0 + Cashback: -200000.0
Total Cost : 103000000.0

==== LOAN + PENALTY + INSURANCE + PROMO ===
Description: Personal Loan (Base Cost: 100000000.0)
+ Ins: 300000.0 + Insurance: 300000.0 + Promo: -10.0%
Total Cost : 106400000.0
```

6. KELEBIHAN DECORATOR PATTERN

- ✓ Menambah behavior tanpa ubah class utama
- ✓ Fleksibel: kombinasi fitur bisa berbeda-beda
- ✓ Mudah dikembangkan (tambah decorator baru)
- ✓ Mengikuti prinsip Open/Closed Principle
- ✓ Cocok untuk fitur opsional / biaya tambahan

7. KAPAN TIDAK MENGGUNAKAN

- ✗ Saat behavior tidak berubah atau tidak opsional
- ✗ Terlalu banyak decorator dapat membuat struktur kompleks dan sulit dibaca
- ✗ Jika semua kombinasi fitur sudah diketahui sejak awal (mungkin lebih baik pakai inheritance)

8. KESIMPULAN

Decorator Pattern sangat cocok digunakan di P2P Lending ketika kita ingin menambahkan fitur opsional seperti penalty, insurance, cashback, promo, atau biaya tambahan lainnya secara dinamis tanpa mengubah class Loan utama. Ini membuat sistem lebih fleksibel, maintainable, dan mudah dikembangkan.



DESIGN PATTERN DI P2P LENDING

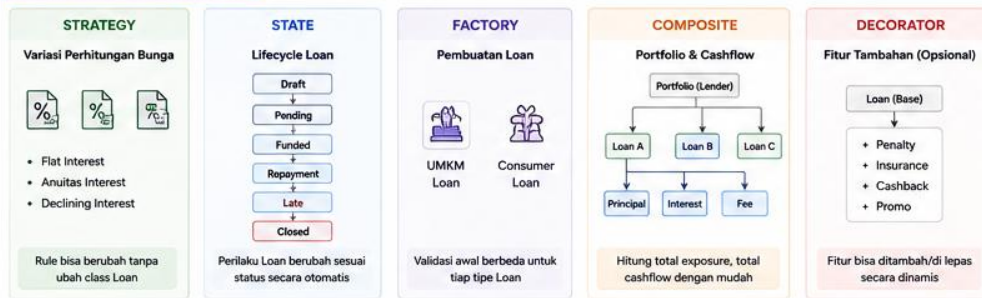
Panduan Lengkap: Kapan Memilih Pattern yang Tepat?



1. PERBANDINGAN CEPAT (DI DOMAIN P2P)

Pattern	Fokus Domain	Kapan Dipakai	Contoh di P2P Lending
Strategy	Variasi rule (bunga, fee, denda) sering berubah	- Algoritma perhitungan sering berubah - Banyak variasi rule berdasarkan kondisi - Perlu mudah extend tanpa ubah code lama	Perhitungan bunga: Flat, Anuitas, Declining Perhitungan fee: Platform Fee, Late Fee
State	Lifecycle loan behavior tergantung status	- Object berubah perilaku sesuai statusnya - Transisi status memiliki rule tertentu - Status dikendalikan dari dalam object	Loan: Draft → Pending → Funded → Repayment → Late → Closed
Factory	Pembuatan object valid enforce invariants	- Banyak tipe object dengan aturan validasi berbeda - Perlu validasi saat pembuatan - Ingin hide detail konstruksi	Buat Loan (UMKM/Consumer) Buat Account (Lender/Borrower) Buat Contract
Composite	Agregasi (portfolio, cashflow) struktur tree	- Struktur data berbentuk hirarki (tree) - Perlu treat single & group secara uniform - Agregasi perhitungan (total, sum, dll)	Portfolio → banyak Loan Cashflow → Principal, Interest, Fee, Penalty
Decorator	Feature tambahan (penalty, promo, insurance) opsional & dinamis	- Fitur opsional yang bisa ditambahkan - Ingin extend behavior tanpa ubah class utama - Bisa stacking banyak fitur tambahan	Tambah Penalty, Insurance, Cashback, Promo/Discount pada Loan

3. CONTOH KASUS DI P2P LENDING



5. CONTOH CODE SINGKAT (JAVA)

```
interface InterestStrategy {
    double calculate(Loan loan);
}

class FlatStrategy implements InterestStrategy {
    public double calculate(Loan loan) {
        return loan.getPrincipal() *
            loan.getRate() * loan.getTenor() / 365.0;
    }
}

interface LoanState {
    void handle(Loan loan);
}

class FundedState implements LoanState {
    public void handle(Loan loan) {
        // logic when loan is funded
        loan.setState(new RepaymentState());
    }
}

class LoanFactory {
    public static Loan createLoan(
        LoanType type, LoanRequest req) {
        if (type == LoanType.UMKM)
            return new UMKMLoan(req);
        else if (type == LoanType.CONSUMER)
            return new ConsumerLoan(req);
        else
            throw new IllegalArgumentException();
    }
}

interface CashFlow {
    double getAmount();
}

class CompositeCashFlow implements CashFlow {
    List<CashFlow> children = new ArrayList<>();
    public void add(CashFlow cf) { children.add(cf); }
    public double getAmount() {
        return children.stream()
            .mapToDouble(CashFlow::getAmount)
            .sum();
    }
}

abstract class LoanDecorator implements Loan {
    protected Loan loan;
    public LoanDecorator(Loan loan) { this.loan = loan; }
    public double getCost() { return loan.getCost(); }
}

class PenaltyDecorator extends LoanDecorator {
    private double penalty;
    public PenaltyDecorator(Loan loan, double penalty) {
        super(loan);
        this.penalty = penalty;
    }
    public double getCost() {
        return loan.getCost() + penalty;
    }
}
```

2. KAPAN MEMILIH PATTERN? (DECISION GUIDE)



4. INSIGHT PENTING (YANG SERING SALAH)



6. KESIMPULAN



KALAU DIGABUNG:



- TIPS PRAKTIK**
- Mulai dari kebutuhan bisnis, bukan pattern
 - Satu masalah → satu pattern utama
 - Jangan over-engineering
 - Selalu pertimbangkan maintainability & testability
 - Pahami domain P2P, baru pilih pattern yang paling pas

